SCALIAN ACADEMY

Cultivons nos talents
Scalian Academy

API ET MISE EN PRODUCTION D'IA



MISE EN PRODUCTION D'UN MODELE

PASSAGE EN PRODUCTION

SCALIAN

Une fois notre modèle entraîné, comment le passer en production ?

Problème :

- **Le « modèle » est un objet complexe**
- **Sa durée de vie du ou des modèles est variable, et peux être difficile à prévoir, et différente de celle du code qui l'entoure**
- **La quasi-totalité des modèles peuvent faire des erreurs**



Résultat:
-D'après Venturebeat 87% des projets de datascience ne sont pas mis en production, 90% selon redapt

LE MACHINE LEARNING EST COMPLEXE

SCALIAN

UNE APPLICATION SERVANT UN MODÈLE :



Data	Model	Code
Schema	Algorithms	Business Needs
Sampling over Time	More Training	Bug Fixes
Volume	Experiments	Configuration

SA MISE EN PRODUCTION AUSSI







Data

Schema

Sampling over Time

Volume

Model



Code

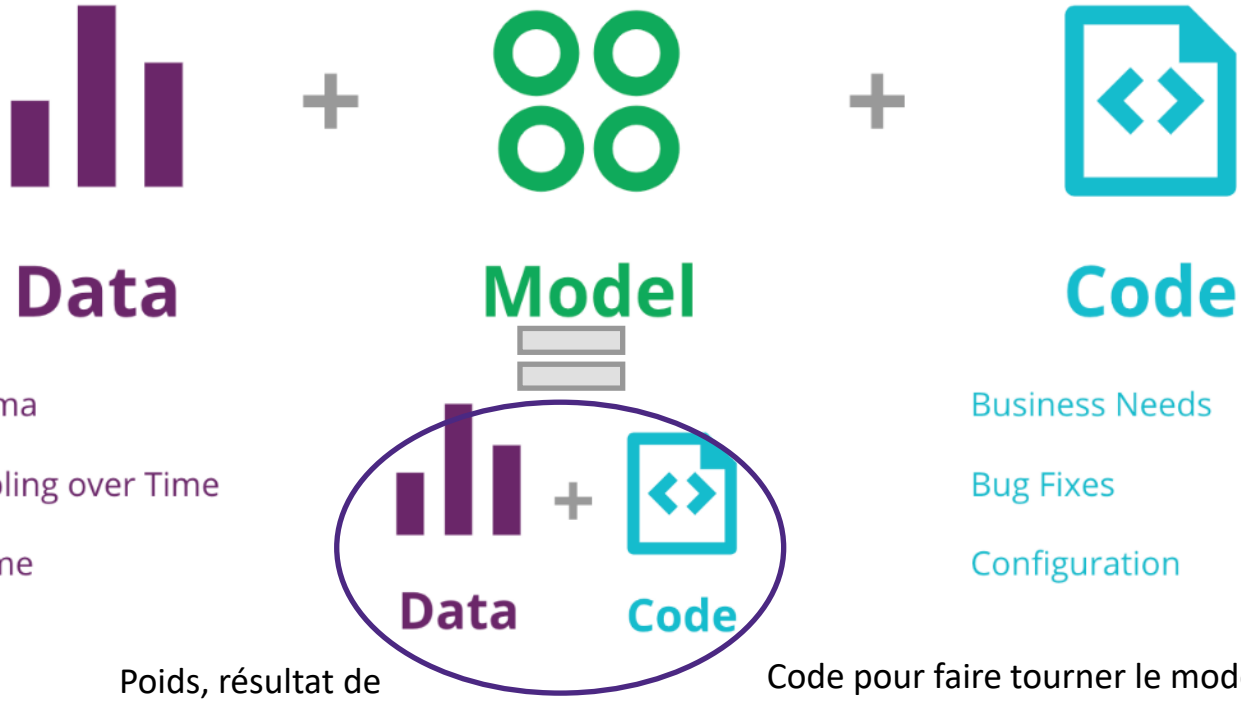
Business Needs

Bug Fixes

Configuration

SA MISE EN PRODUCTION AUSSI

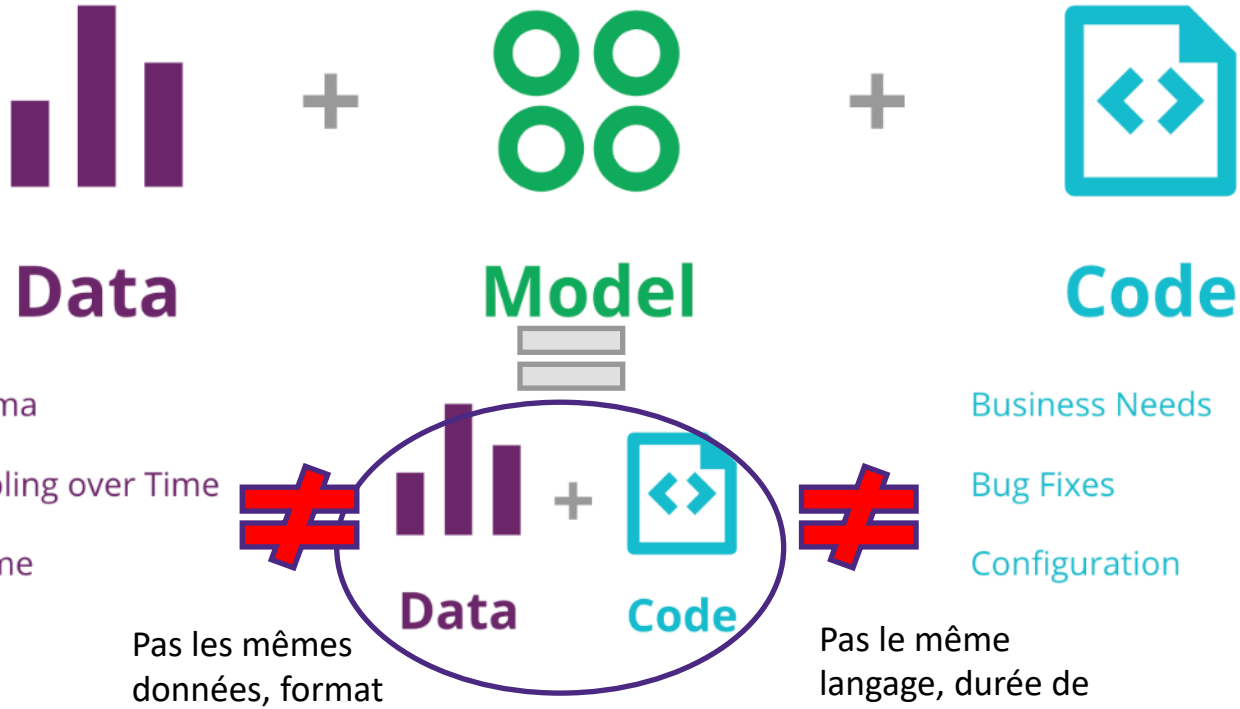
SCALIAN



Poids, résultat de l'apprentissage
1 Ko à >1 Tera

Code pour faire tourner le modèle (scikit-learn, architecture pytorch, preprocessing, …)
Vide à un projet à part entière

SA MISE EN PRODUCTION AUSSI



SA MISE EN PRODUCTION AUSSI

LE MODÈLE, UN COMPROMIS DIFFÉRENT ENTRE CODE ET DATA

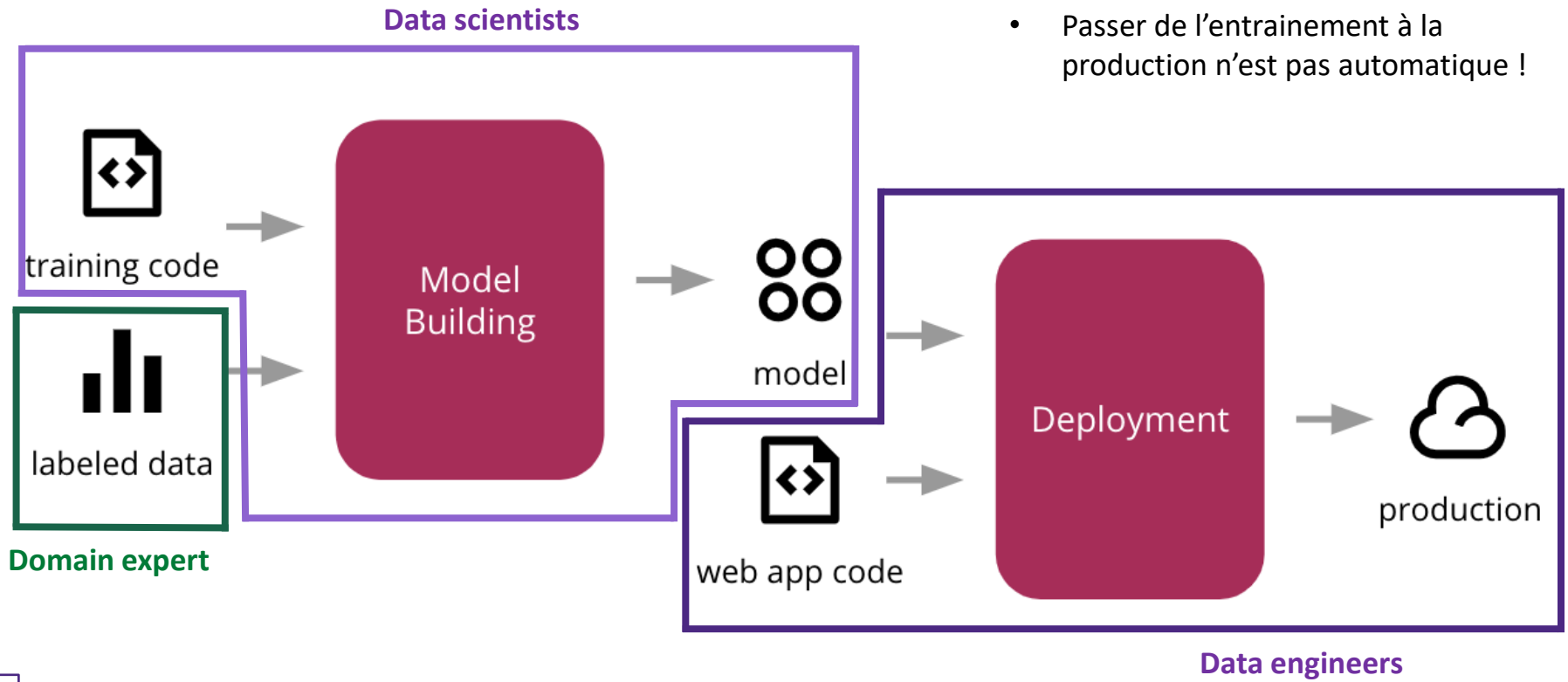


Model



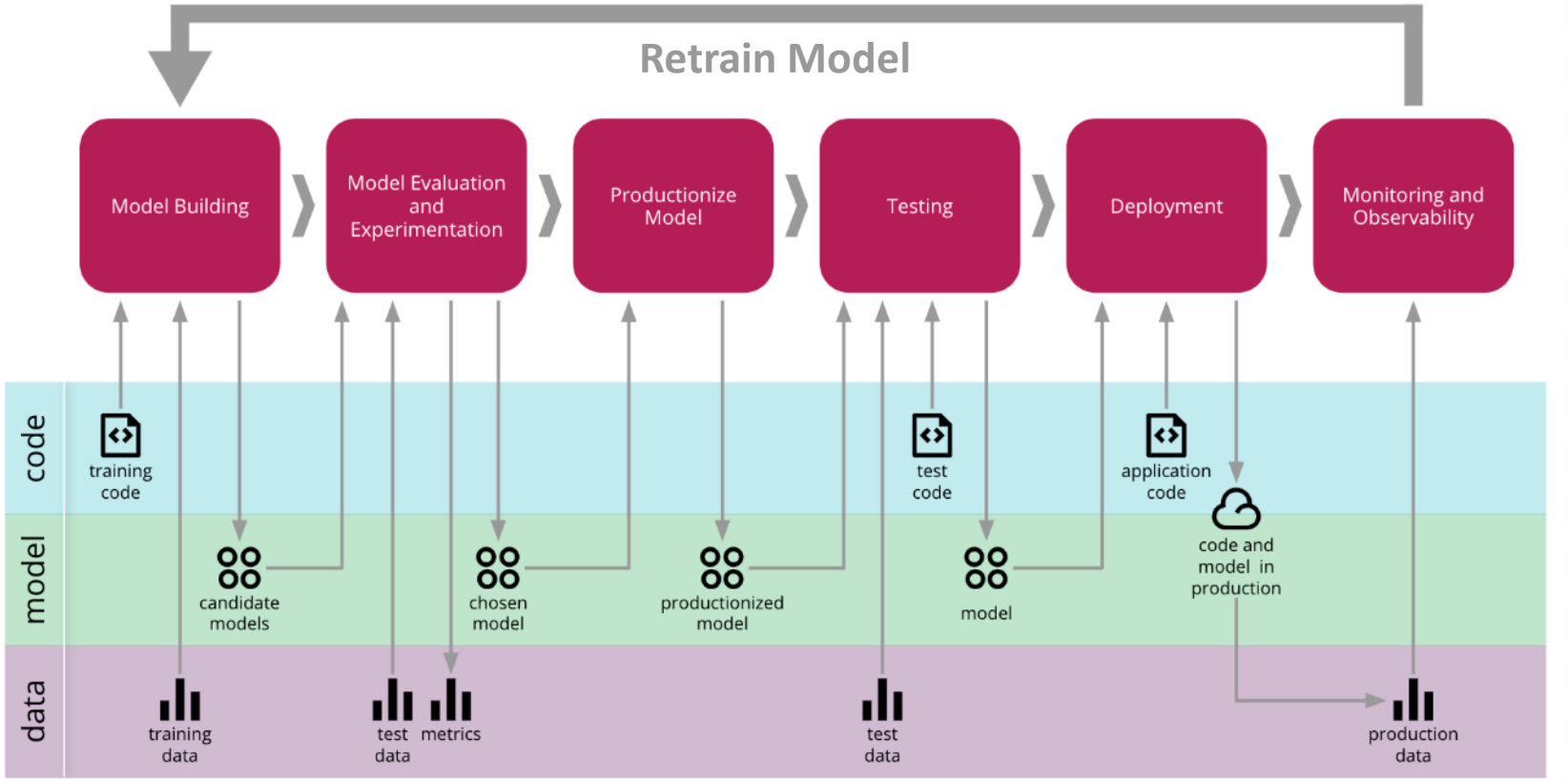
LES SILOS D'UN PIPELINE

SCALIAN



- Passer de l'entrainement à la production n'est pas automatique !

PIPELINE COMPLET



LES TESTS DE MACHINE LEARNING

LES TESTS SUPPLÉMENTAIRES PAR RAPPORT À UN PROJET CLASSIQUE

Tests de performance

Vous les connaissez !

Valider la non regression

Vérifier que la performance du modèle une fois déployé est la même qu'en sortie de l'entrainement -> jeu de test

Domain Gap

Vérifier que les données de productions ne sont pas différentes des données utilisées pour les jeux d'entrainement, validation et test

Biais et éthique

Vérification et si possible explication des décisions du modèles par rapport à des populations qui pourraient présenter des biais. Peut être une ***contrainte reglementaire*** (IA act).

Drift detection

Vérifier que les performances du modèle ne dérivent pas au court du temps

Résistance aux attaques

Vérifier qu'un utilisateur ne puisse pas détourner (par exemple avec du prompt engineering) le modèle pour des utilisations dangereuses ou non voulue (fuite des données d'entrainement, demander à créer un virus, …). Se vérifie par des outils, modèles ou jeu de données spécifiques.

MISES À JOUR DE MODÈLE(S)

QUELQUES PRATIQUES

Shadow model
Permet de valider en production la bonne performance d'un nouveau modèle (en shadow) le temps des tests

Multiple modeles
Faire un model par tâche plutôt qu'un modèle sur plusieurs tâches
ex: détection de produit inappropriée:
(si une catégorie change, moins d'impact)

source

Online learning
Les modèles s'entraînent au fil de l'eau en production

Nécessite de vérifier en permanence le comportement du modèle

MISE EN SERVICE D'UN MODÈLE

3 STRATÉGIES

Embedded model

Le modèle est buildé et déployé au sein de l'application

- Performant (sous conditions)
- Rend le remplacement d'un modèle lourd
- Nécessite que les hardwares sur lesquels tournent les applications soient suffisamment performants pour le modèle
- Nécessite que les langages soient les mêmes ou compatibles

Model as a service

Le modèle est déployé via un serveur externe à l'application

- Peu engendrer des latences lors des inférences
  - Souvent négligeables par rapport au temps de traitement du modèle
- Scalable facilement
- Permettrent de rendre indépendant les contraintes entre partie applicative et modèle (langages, hardware, compétences des développeurs, durée de vie …)

Model as a data

Le modèle est traité indépendamment et passé en paramètre lors du runtime

- Pas mutuellement exclusif avec les stratégies précédentes
- Permet de facilement remplacer/gérer/maj des modèles
- Certains des modèles sont uniquement des data à fournir à un framework existant

MISE EN SERVICE D'UN MODÈLE

UNE MAJORITAIRE

Embedded model

Le modèle est buildé et déployé au sein de l'application

- Performant (sous conditions)
- Rend le remplacement d'un modèle lourd
- Nécessite que les hardwares sur lesquels tournent les applications soient suffisamment performants pour le modèle
- Nécessite que les langages soient les mêmes ou compatibles

Model as a service

Le modèle est déployé dans un service spécifique séparé du reste de l'application

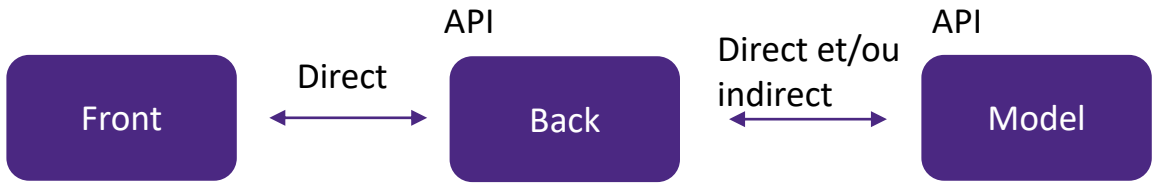
- Peu engendrer des latences lors des inférences
  - Souvent négligeables par rapport au temps de traitement du modèle
- Scalable facilement
- Permettrent de rendre indépendant les contraintes entre partie applicative et modèle (langages, hardware, compétences des développeurs, durée de vie …)

Model as a data

Le modèle est traité indépendamment et passé en paramètre lors du runtime

- Pas mutuellement exclusif avec les stratégies précédentes
- Permet de facilement remplacer/gérer/maj des modèles
- Certains des modèles sont uniquement des data à fournir à un framework existant

MODEL AS A SERVICE



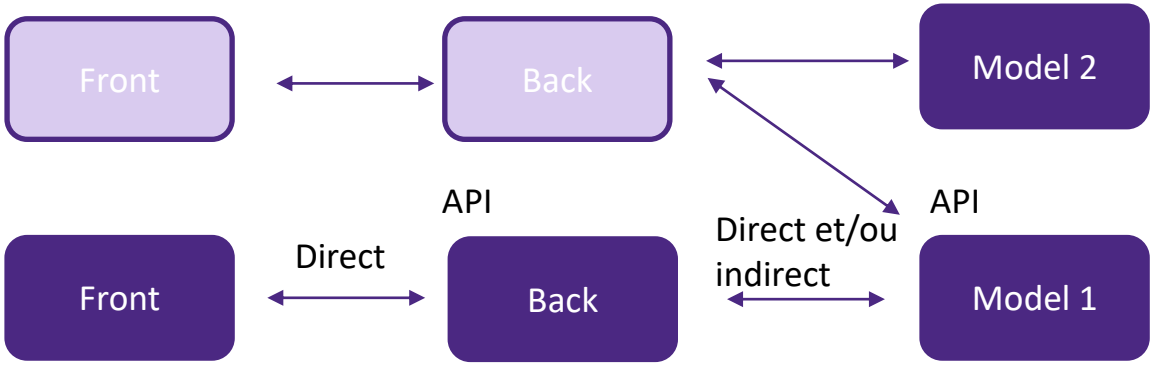
Interface utilisateur

Application
- Gestion des utilisateurs
- Fonctionnement de l'app
- Base de données

….

Traitement IA
- Reçoit des données ou des liens vers celles-ci.
- Renvoit l'information calculée dessus ou des liens vers ceux-ci.

MODEL AS A SERVICE



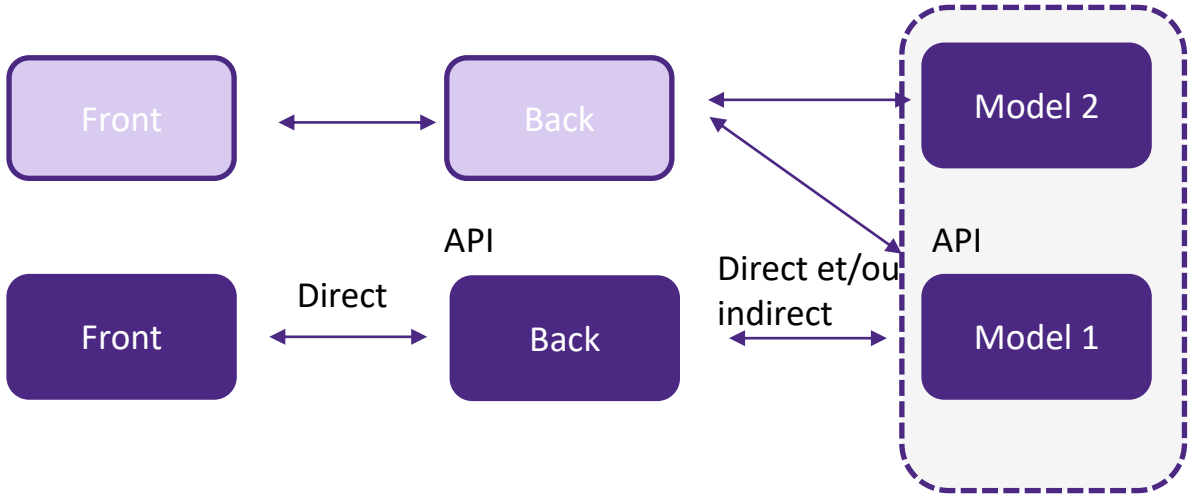
Interface utilisateur

Application
- Gestion des utilisateurs
- Fonctionnement de l'app
- Base de données

….

Traitement IA
- Reçoit des données ou des liens vers celles-ci.
- Renvoit l'information calculée dessus ou des liens vers ceux-ci.

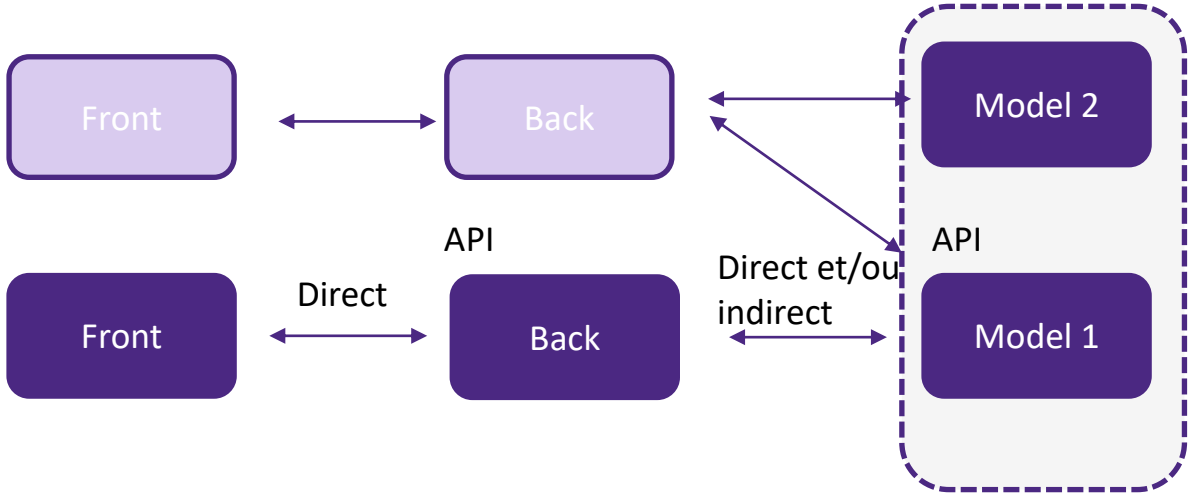
MODEL AS A SERVICE



« IA Layer » / « Model Layer »

Contraintes matérielles, technologiques, CI/CD, Compétence différentes.

MODEL AS A SERVICE



« IA Layer » / « Model Layer »

- **Il n'y a pas une seule bonne architecture. Je montre ici un exemple (souvent applicable), mais à appliquer, modifier, en fonctions des contraintes et de la complexité réelles.**
- **Il faut savoir implémenter des APIs, back ou IA.**

LES API

1- QUELQUES RAPPELS

QUELQUES RAPPELS

SCALIAN

- **API :** application programming interface -> moyen de communication entre deux logiciels
  *=> L'objectif est de fournir une interface **standardisée** et **réutilisable** de communication entre deux applications via un reseau.*

- **RESTful vs SOAP :** ***Re**presentational **S**tate **T**ransfer*
  => ce sont des ensembles de normes de code et d'architecture qui structurent la façon de communiquer de l'application
  => REST se base sur le protocole HTTP pour transférer des informations

Les lignes directrices d'une API REST

1. Séparation client serveur
2. Stateless : le serveur ne sauvegarde aucune requête ou résultat de requête précédente (il faut passer toutes les données caractérisant le contexte de votre requête).
3. Cacheable : la réponse doit contenir l'information sur la capacité du client à mettre les données en cache (n° de version etc)
4. Uniform interface: l'api constitue un contrat entre client et le serveur
5. Layered system : le client ne sait pas ce qu'il se passe derrière l'API
6. Code on demande: capacité du serveur à envoyer du code au client à la demande pour pouvoir étendre une fonctionnalité

Ressources complémentaires : openclassroom / red hat documentation

LES API

2- DÉVELOPPER AVEC FLASK

STREAMLIT



Streamlit

- Pas adapté à la production
- Pas de vrai séparation front/back
- Le script python est relu en entier à chaque requête (rapidement source de latence).
- Avec les progrès des LLMs, l'intérêt diminue même pour un Poc rapide.

FLASK



- Crée en 2010
- Communauté large et active
- Petit et efficace
- Synchronous by design (WSGI)
  - flask[async] : compatibilité, mais pas performance.
- Nu de base mais possibilité de rajouter
- Complexe à maintenir

FASTAPI



- Crée en 2018
- Construit à partir de Starlette (framework python web) et Uvicorn (serveur http python)
- Asynchrone : implémente ASGI
- Temps de réponse jusqu'à 3x plus rapide que Flask (égal à go ou node)
- Documentation swagger générée automatiquement
- En python, ce qui permet d'inclure des dépendances IA facilement (preprocessing, langchain, vecteor search, …)

POURQUOI FastAPI

SCALIAN

- L'asynchrone en IA, c'est comme un fastfood
  - La cuisine (l'IA) **prend du temps** et **couteux à paralléliser** (on ne va pas acheter un four par client parallèle)
  - Le caissier (l'API) ne doit pas bloquer la file d'attente de client en attendant que le plat du premier client soit préparé.
    - Sinon il faut un caissier par client.
  - En asynchrone, le caissier peut prendre une commande, l'envoyer aux cuisines, puis passer au second client, et rappeler le premier quand la cuisine lui envoit le plat.

- Python peut rendre le temps de développement plus rapide :
  - certains traitements annexes à l'IA doivent parfois être effectués dans l'API.
  - Dépendance à des bilbiothèques pour parser et présenter les données telles que l'attend le modèle.
  - Peut séparer API back en api back « applicative » et api back « IA »
    - Front <-> API back <-> API back IA <-> Modele

STRUCTURER FastAPI

SCALIAN

Pas de règle ou consensus universel. C'est **ma version** que je présente ici.

Deux types très différents d'architectures, deux structures différentes.

Architecture micro-service.

FastAPI a **une seule responsabilité.**

Ex: Exposer une base de donnée.

Ex: Wrapper sur un modèle ou une technologie.

Deux périmètres -> Deux fastAPI

+ : Simple à écrire

+ : Garantie une séparation et scalabilité indépendante.

- : Coûteux (cout de transfert entre les services...)

- : Une partie de la complexité est déplacée sur la couche réseau et la CI-CD.

Architecture macro-service.

FastAPI peut gérer plusieurs fonctionnalités

Ex: Accès à des LLMs et des embeddings

Ex: Gérer des agents IA (demande d'accéder à modèles, bases de données, vector store, …)

Ex: Dépendances supplémentaires liées à l'authentification, le contrôle des accès, le monitoring, l'administration.

Pas forcément monolithique.

+ : Efficace, pas de temps de transfert entre services

- : La garantie de séparation et d'évolution doit être portée par la structure du code

STRUCTURER FastAPI

SCALIAN

Micro-service

```
app/
├── __init__.py
├── main.py
├── core/
│   ├── __init__.py
│   └── config.py
├── api/
│   ├── __init__.py
│   ├── v1/
│   │   ├── __init__.py
│   │   └── _api.py
│   └── v2/
│       ├── __init__.py
│       └── _api.py
├── routers/
│   ├── __init__.py
│   ├── _auth.py
│   └── _person.py
├── schemas/
│   ├── __init__.py
│   └── _person.py
├── cruds/
│   ├── __init__.py
│   └── _person_crud.py
└── services/
    ├── __init__.py
    └── _person_service.py
```

<- parties nécessaires

<- code commun entre les modules

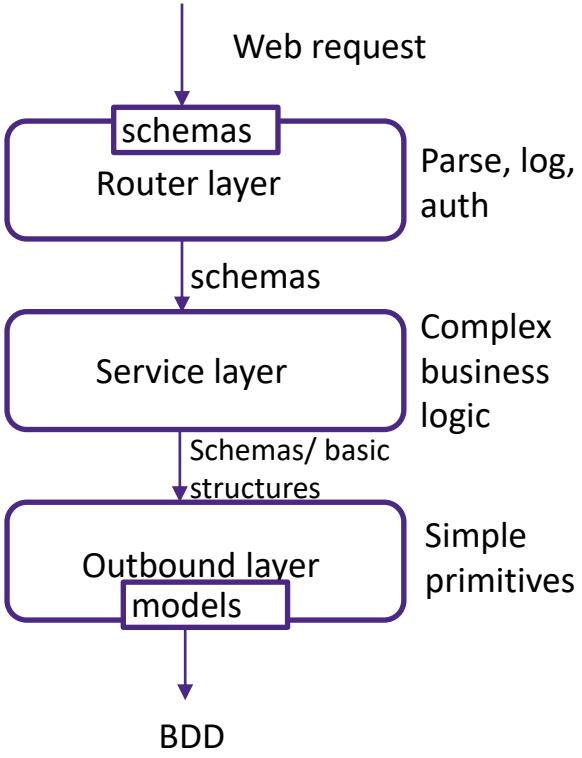
<- simple agrégateur:
permet de spécifier les endpoints accessibles en v1, v2, dev, prod,…

<- endpoints

<- api schemas

<- CRUD et bdd models

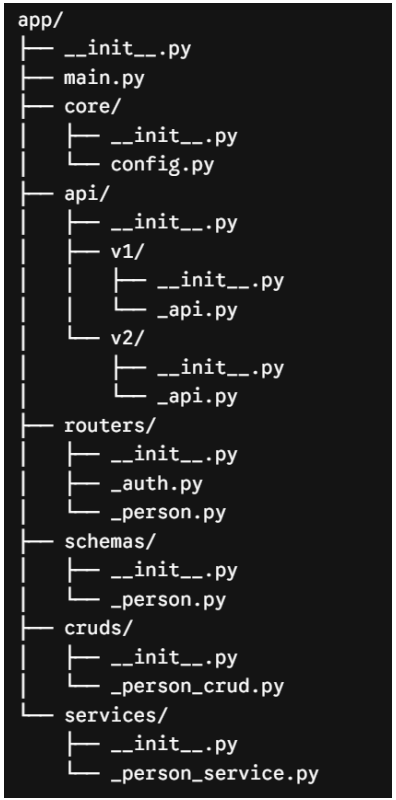
<- service



STRUCTURER FastAPI

SCALIAN

Micro-service



<- parties nécessaires

<- code commun entre les modules

<- simple agrégateur:
permet de spécifier les endpoints accessibles en v1, v2, dev, prod,…

<- endpoints

<- api schemas

<- CRUD et bdd models

<- service

Des patterns classiques:

- depends()
  - Database session, client
  - Authentification, contrôle des droits
- UoW (Unit of Work):
  - Traitements garantis au début et/ou à la fin de chaque service (par exemple push sur la database, plutôt qu'éparpillés dans le crud)
- DTO : séparation claire des objets entre les couches. Il ne faut pas qu'une modification de l'entrée de l'api casse le modele de données.

STRUCTURER FastAPI

SCALIAN

Macro Service

```
app/
├── __init__.py
├── main.py
├── core/
│   ├── __init__.py
│   ├── router.py
│   ├── service.py
│   ├── schemas.py
│   └── internal/
│       ├── __init__.py
│       ├── config.py
│       ├── database.py
│       ├── security.py
│       └── exceptions.py
├── api/
│   ├── __init__.py
│   └── v1/
│       ├── __init__.py
│       └── router.py
```

<- code commun entre les modules

<- simple agrégateur:
permet de spécifier les endpoints accessibles en v1, v2, dev, prod,…

```
├── blob_storage/
│   ├── __init__.py
│   ├── router.py
│   └── service.py
├── iotedge/
│   ├── __init__.py
│   ├── router.py
│   └── service.py
├── inference_server/
│   ├── __init__.py
│   ├── router/
│   │   ├── __init__.py
│   │   ├── _inference.py
│   │   └── _registry.py
│   ├── service.py
│   ├── crud.py
│   └── schemas.py
└── pipeline/
    ├── __init__.py
    ├── router.py
    └── service.py
```

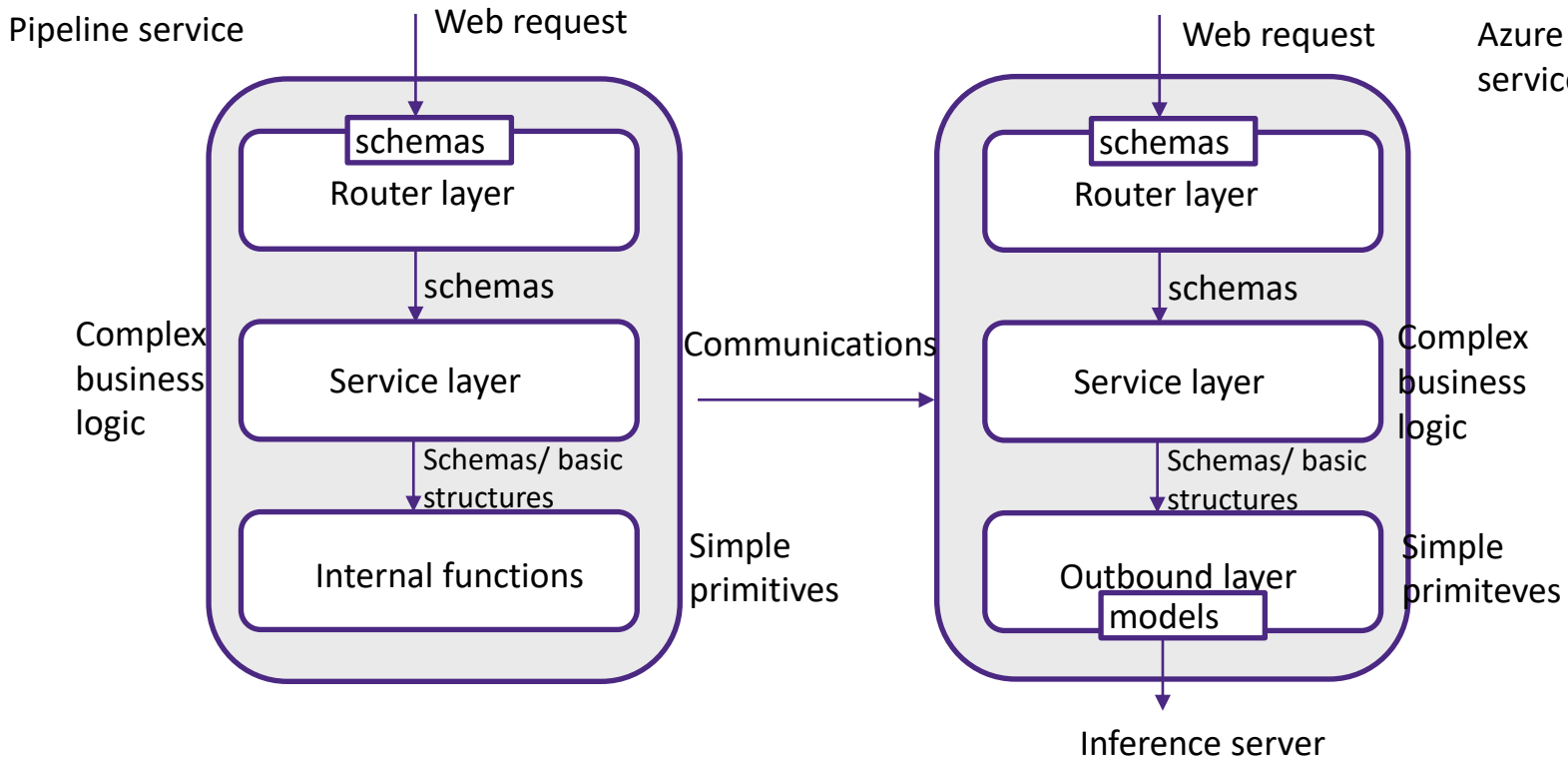
Chaque service a sa propre couche service, router, primitives.

- simple .py si petit
- Sous-modules s'ils grossissent (avec déclaration dans l'_init__ pour être invisible).

STRUCTURER FastAPI

SCALIAN

Macro Service



STRUCTURER FastAPI

Macro Service

Objectifs :

- **Séparer le code, éviter le sac de nœuds.**
- **Faciliter la compréhension**
  - Tous les services ont la même structure
  - Même une IA rangera au bon endroit (probablement).
- Être robuste à des ajouts de fonctionnalités.
- Remplacer une technologie, (ex: un serveur d'inférence), n'impacte pas les autres services.

- Inspiré de *https://github.com/zhanymkanov/fastapi-best-practices*

STRUCTURER FastAPI

SCALIAN

Exemples de fonctionnalités communes

- Routes de monitoring

```python
# Import
import prometheus_client

# Route de métriques
@router.get("/metrics")
def get_metrics():
    return prometheus_client.generate_latest()
```

- Routes de healthcheck

```python
@router.get("/healthz", status_code=status.HTTP_200_OK)
def healthcheck():
    return {"status": "ok"}
```

- « Depends » pour l'authentification

```python
async def get_current_user(token: str = Depends(oauth2_scheme)) -> User:
    return security.verify_access_token(token)
```

STRUCTURER FastAPI

SCALIAN

Exemples de fonctionnalités communes (2)

- Initialisation et extinction de l'application

```python
@asynccontextmanager
async def lifespan(app: FastAPI):
    # 1. TACHE D'INIT (exécutée au lancement de l'application)
    print("--- [STARTUP] Connexion DB, chargement des modèles ML, etc. ---")

    yield  # L'application tourne et traite les requêtes ici

    # 2. TACHE DE FINALISATION (exécutée à l'arrêt de l'application)
    print("--- [SHUTDOWN] Fermeture des connexions, nettoyage ---")

app = FastAPI(lifespan=lifespan)
```

- Middleware

```python
from fastapi.middleware.cors import CORSMiddleware

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

- Variable d'environnements ou paramètres spécifiées dans un module dédié (fast-crashing).